SpamGuard AI — Software Requirements Specification (SRS)

Document Reference: `SG-SRS-IA3-V1.0`
Evaluation: Software Engineering Internal Assessment 3 (IA3)
Academic Micro-Project: Micro Project – 05 (Semester 3)
Domain: Artificial Intelligence / Cybersecurity / Email Threat Intelligence
Author / Team: Candidate / SpamGuard AI Development Team
Date of Release: October 2026
Status: Completed, Verified, Implementation-Aligned

Executive Summary & Metadata

SpamGuard AI is a production-grade, highly interpretable electronic mail threat classification and intelligence system. It utilizes classical machine learning—specifically Term Frequency-Inverse Document Frequency (TF-IDF) unigram/bigram feature extraction paired with an optimized Multinomial Naïve Bayes probabilistic classifier ($\alpha = 0.1$)—trained and evaluated on 5,923 authentic MIME messages from the official Apache SpamAssassin Public Corpus.

This Software Requirements Specification (SRS) establishes the definitive functional, non-functional, interface, data, and behavioral requirements for SpamGuard AI, directly aligned with the verified software implementation across its Next.js frontend (port 3000), FastAPI REST inference service (port 8000), Streamlit fallback dashboard (port 8501), and frozen core inference engine.

---

1. Introduction

1.1 Purpose

This document establishes the comprehensive software requirements for **SpamGuard AI — AI-Based Spam Email Classifier**. It details all functional capabilities, performance criteria, security boundaries, external interface protocols, data schemas, and use-case interactions realized in the working software system. This specification serves as the formal baseline for the Software Engineering IA3 evaluation, software testing, quality verification, and academic auditing.

1.2 Scope

SpamGuard AI delivers an end-to-end, locally hosted email threat intelligence platform designed to classify incoming emails into **Ham (Legitimate, 0)** or **Spam (Malicious / Unsolicited, 1)** with high precision and sub-millisecond execution latency.

The system encompasses: 1. **Core Inference Engine (`src/predict.py`):** Encapsulating frozen model loading, pre-inference SHA-256 cryptographic integrity verification, text normalization, vectorized single/batch classification, mathematical feature attribution via Naïve Bayes log-odds difference, and spreadsheet formula injection defense. 2. **RESTful Microservice (`src/api.py`):** High-performance FastAPI service on port 8000 exposing typed Pydantic endpoints for prediction, bulk CSV ingestion, verified academic telemetry, system health monitoring, and Google OAuth 2.0 session handling. 3. **Primary Web Presentation Layer (`frontend/`):** Contemporary Next.js 16 + TypeScript single-page application on port 3000 styled with an editorial Anthracite dark-mode design, dynamic routing, real-time probability visualization, and interactive inspection drawers. 4. **Resilient Application Fallback (`app.py`):** Fully functional, standalone Streamlit dashboard on port 8501 providing complete operational redundancy, Plotly telemetry,

and educational disclosures without external framework dependencies. 5. **Threat Intelligence Integration (`src/gmail/`):** Optional Google OAuth 2.0 integration operating under strict read-only scopes (`gmail.readonly`) allowing security analysts to inspect real mailboxes without storing message text on disk.

1.3 Project Objectives

SpamGuard AI fulfills the following core engineering and pedagogical objectives: - **Objective 1 (Ingestion & Sanitization):** Parse authentic multipart MIME messages from raw archives while rejecting directory traversal, symlink attacks, and executable attachments. - **Objective 2 (Data Leakage Elimination):** Enforce strict stratified partitioning (70% Train, 15% Validation, 15% Test) ensuring zero content overlap and zero shared Message-IDs across partitions. - **Objective 3 (Feature Representation):** Extract high-dimensional textual feature representations using sublinear unigram/bigram TF-IDF vectorization capped at 20,000 features. - **Objective 4 (Probabilistic Modeling):** Optimize Lidstone smoothing parameter α against a dedicated validation set, achieving optimal F1-score performance. - **Objective 5 (Asymmetric Error Control):** Suppress the False Positive Rate (FPR) below 2.0% on held-out test data to prevent legitimate correspondence from being erroneously flagged. - **Objective 6 (Explainability by Design):** Formulate mathematical token-level log-odds attribution providing clear human-interpretable rationale for every prediction. - **Objective 7 (Defensive Hardening):** Guard against CSV formula injection attacks, payload denial-of-service, CSRF state tampering, and unauthorized model mutations. - **Objective 8 (Full Redundancy):** Provide a decoupled Next.js/FastAPI production architecture with a 100% independent Streamlit fallback.

1.4 Intended Audience

- **Software Engineering Academic Evaluators:** Reviewing specification rigor, UML alignment, verification test cases, and requirement traceability.
- **Cybersecurity & Machine Learning Engineers:** Auditing algorithm selection, explainability formulas, and defensive attack mitigations.
- **System Administrators & DevOps:** Configuring operating environments, port bindings, and API dependencies.

1.5 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
API	Application Programming Interface (HTTP/REST communication)
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
CSV	Comma-Separated Values file format
DoR / DoS	Denial of Service
FNR	False Negative Rate ($\mathrm{FN} / (\mathrm{TP} + \mathrm{FN})$)
FPR	False Positive Rate ($\mathrm{FP} / (\mathrm{TN} + \mathrm{FP})$)
Ham	Legitimate, desired electronic mail correspondence (Class 0)
MIME	Multipurpose Internet Mail Extensions
MNB	Multinomial Naïve Bayes
NFR	Non-Functional Requirement
NFKC	Unicode Normalization Form KC (Compatibility Decomposition, followed by Canonical Composition)
OAuth 2.0	Open Authorization 2.0 protocol for secure third-party delegated access
RTM	Requirement Traceability Matrix
SHA-256	Secure Hash Algorithm 256-bit cryptographic digest
Spam	Unsolicited, commercial, fraudulent, or malicious email correspondence (Class 1)
SPA	Single Page Application
SRS	Software Requirements Specification
TF-IDF	Term Frequency-Inverse Document Frequency

---

2. Overall Description

2.1 Product Perspective

SpamGuard AI is a self-contained, multi-tier threat classification platform designed for local enterprise network deployment. Unlike commercial cloud-based email filters that require forwarding private corporate emails to external third-party APIs (e.g., OpenAI or Google Gemini), SpamGuard AI operates entirely on-premise using a frozen, locally serialized Scikit-learn pipeline. All inference is computed in local volatile memory, guaranteeing absolute data privacy and zero cloud egress.



2.2 User Classes and Characteristics

1. **Tier 1 — Security Analyst / Enterprise User:**
2. Evaluates suspicious emails, reviews threat verdicts, analyzes token log-odds attributions, uploads bulk CSV logs from mail transfer agents (MTAs), and exports sanitized classification reports.
3. Requires an intuitive, low-friction interface with clear visual threat indicators and sub-second feedback.
4. **Tier 2 — Academic Evaluator / Auditor:**
5. Inspects model architecture, validates held-out test metrics (accuracy, precision, recall, F1, ROC-AUC), audits confusion matrix distributions, and verifies mathematical rigor.
6. Requires access to verified academic reports, confusion matrix breakdowns, and training summaries.
7. **Tier 3 — System Administrator / DevOps:**
8. Deploys the service, validates API health (`/api/health`), audits cryptographic SHA-256 model checksums, monitors CPU/memory latency, and configures OAuth credentials.

2.3 Operating Environment

- **Operating Systems:** Windows 10/11 (verified on build 26300), Ubuntu Linux 22.04 LTS / 24.04 LTS, macOS Sonoma / Sequoia.
- **Python Environment:** Python 3.12.x running within a dedicated virtual environment (`.venv`).
- **Node.js Environment:** Node.js v18.0.0 or higher (v20+ recommended) with npm 10+.
- **Network Ports:** Port 3000 (Next.js frontend), Port 8000 (FastAPI REST service), Port 8501 (Streamlit fallback).
- **Client Web Browsers:** Google Chrome 120+, Mozilla Firefox 120+, Microsoft Edge 120+, Apple Safari 17+.

2.4 Assumptions and Dependencies

- **Model Availability:** The pre-trained pipeline artifact `models/spam_pipeline.joblib` exists, matches the expected SHA-256 hash `c652119c99f0db2abe0788969d002818478bb63be95e31e5f337eda8da6ddbb0`, and contains fitted `tfidf` and `classifier` steps.
- **Hardware Resources:** Minimum 4 GB of host RAM and modern multi-core CPU. Zero requirement for discrete GPU hardware.
- **Software Dependencies:** `scikit-learn==1.9.1`, `fastapi==0.162.0`, `uvicorn==0.54.0`, `pandas==3.1.0`, `numpy==2.2.6`, `joblib==1.6.0`, `streamlit==1.66.1`, `plotly==7.1.0`, `next==16.4.3`, `react==19.3.0`.

2.5 Constraints

- **Model Immutability:** The machine learning classifier is completely frozen. No online learning, re-fitting, or weight adaptation occurs during inference or application lifecycle.
- **Zero Disk Persistence of Message Bodies:** User emails analyzed via single scan or batch classification are processed in volatile memory only and must never be written to persistent log files or relational databases.
- **Payload Limits:** Single email classification requests are capped at 1,000,000 characters (1 MB). Batch CSV uploads are limited to 10,000 rows.

---

3. Functional Requirements

FR-001: Email Subject and Body Entry

- **Description:** The system shall provide distinct input fields for email Subject and email Body in both the Next.js and Streamlit user interfaces.
- **Inputs:** Text strings for Subject line (optional) and Body text (optional, but at least one must be non-empty).
- **Processing:** The system shall combine both fields following the convention: `"{subject}\n\n{body}"` if both are present, or the individual non-empty string.
- **Output:** Unified raw text buffer ready for sanitization.

FR-002: Synthetic Demonstration Sample Selection

- **Description:** The system shall provide a curated library of pre-configured demonstration emails representing diverse threat and legitimate profiles (e.g., Urgent Phishing Wire Transfer, Prize Lottery Scam, Systems Engineering Update, Academic Meeting Notice).
- **Processing:** Selecting a sample shall immediately populate the Subject and Body input fields.
- **Output:** Pre-filled input fields allowing instant classification without manual typing.

FR-003: Deterministic Text Preprocessing and Normalization

- **Description:** The system shall sanitize incoming text identically to the training pipeline using Unicode NFKC normalization, whitespace compression, lowercasing, and regex token cleaning.
- **Validation:** If the cleaned text is empty or contains only whitespace, the system shall reject the request with a `SpamGuardValidationError` and return HTTP 422.
- **Output:** Cleaned, normalized string matching the TF-IDF feature space.

FR-004: Binary Threat Classification (Spam vs. Ham)

- **Description:** The system shall classify the normalized email text into binary categories: **Ham (0)** or **Spam (1)** using the frozen Multinomial Naïve Bayes model.

- **Processing:** Apply `pipeline.predict([cleaned_text])`.
- **Output:** Categorical label (`"Spam"` or `"Ham"`) and integer class (`1` or `0`).

FR-005: Calibrated Probability Estimation & Visual Telemetry

- **Description:** The system shall compute posterior class probabilities $P(\text{Ham})$ and $P(\text{Spam})$ using `pipeline.predict_proba()`.
- **Validation:** Probabilities must be strictly finite real numbers in the range $[0.0, 1.0]$ summing to 1.0 ± 10^{-5}.
- **Output:** Floating-point probabilities rendered via animated gauges and numerical percentage badges.

FR-006: Mathematical Feature Contribution Explainability

- **Description:** When explainability is enabled (`explain=True`), the system shall calculate the token-level contribution towards the classification verdict using the Naïve Bayes log-odds difference:

$$\Delta_odds(w) = \log P(w \mid Spam) - \log P(w \mid Ham)$$

$$\text{Contribution}(w) = \text{TF-IDF}(w) \times \Delta_odds(w)$$

- **Processing:** Tokens with $\Delta_{odds} > 0$ shall be tagged `"Spam-leaning"`; tokens with $\Delta_{odds} \leq 0$ shall be tagged `"Ham-leaning"`. The top k tokens (default 5 or 10) shall be extracted.
- **Output:** Structured list of `FeatureContribution` objects rendered as a color-coded polarity impact table.

FR-007: Batch CSV File Upload and Schema Validation

- **Description:** The system shall allow users to upload CSV files containing multiple emails for high-throughput batch classification.
- **Validation:** The CSV must contain either:
- Both a `'subject'` and `'body'` column (case-insensitive), OR
- A unified `'text'` / `'message'` column. If neither schema is satisfied, the system shall reject the upload with an explicit schema error message.
- **Row Boundary:** The system shall reject CSV files containing more than 10,000 rows to prevent memory exhaustion.

FR-008: High-Throughput Vectorized Batch Inference

- **Description:** The system shall perform batch classification using vectorized Scikit-learn array operations rather than iterative row-by-row loops.
- **Processing:** Batch text preparation, vectorization, and inference executed as contiguous NumPy arrays.
- **Output:** Comprehensive summary statistics: total emails scanned, spam count, ham count, spam percentage, ham percentage, and overall processing latency.

FR-009: Batch Result Filtering and Dynamic Search

- **Description:** The system shall allow users to dynamically filter batch classification results by verdict: `"All"`, `"Spam Only"`, `"Ham Only"`, or `"Needs Review"`.
- **Processing:** Client-side and in-memory filtering executed instantaneously without triggering re-inference.
- **Output:** Filtered data table displaying email identifier, preview, predicted verdict, and spam probability.

FR-010: Spreadsheet Formula Injection Sanitization & CSV Export

- **Description:** The system shall allow users to export classified batch results as a downloadable CSV file while enforcing automated defense against CSV formula injection (OWASP CSV Injection).
- **Processing:** Every text cell shall be passed through `sanitize_csv_cell()`. If a cell begins with `'='`, `'+'`, `'-'`, `'@'`, `'\t'`, `'\r'`, or `'\n'`, the system shall prepend a single-quote character (`'`) to neutralize dynamic formula execution in Microsoft Excel and LibreOffice Calc.
- **Output:** Sanitized, downloadable CSV stream.

FR-011: Model Performance Analytics & Empirical Disclosures

- **Description:** The system shall display verified academic performance metrics computed strictly on the held-out, quarantined test partition ($n=889$).
- **Metrics Presented:**
- Accuracy: 97.30%
- Spam Precision: 97.33%
- Spam Recall: 93.75%
- Spam F1-Score: 95.51%
- ROC-AUC: 0.9965
- False Positive Rate: 1.13% (7 / 617 legitimate emails)
- Confusion Matrix: $\text{TN} = 610$, $\text{FP} = 7$, $\text{FN} = 17$, $\text{TP} = 255$.
- **Comparative Baseline:** Direct comparison against the Zero-Intelligence Majority-Class Baseline (Accuracy 69.40%, Precision 0.0%, F1 0.0%).

FR-012: RESTful API Health & Cryptographic Telemetry

- **Description:** The FastAPI service shall expose a `GET /api/health` endpoint providing operational status, active model identifier, Python/OS platform version, and real-time verification of the model's SHA-256 checksum against `EXPECTED_PIPELINE_SHA256`.
- **Output:** JSON object conforming to `SystemHealthResponse`.

FR-013: RESTful Inference Endpoints

- **Description:** The FastAPI backend shall expose typed REST endpoints:
- `POST /api/predict`: Single email classification with optional explainability.
- `POST /api/predict/batch`: Bulk JSON array classification.
- `POST /api/predict/csv`: Multipart form-data CSV file upload.
- `GET /api/metrics`: Held-out test evaluation figures and confusion matrix data.
- `GET /api/samples`: Synthetic demonstration sample catalog.

FR-014: Dual Presentation Architecture & Resilient Fallback

- **Description:** The system shall support simultaneous or independent operation of both:
- The Next.js 16 modern editorial frontend on port 3000.
- The Streamlit 1.66 cyber-dashboard on port 8501. Both interfaces must interface with the same frozen model engine and produce bit-for-bit identical classifications.

FR-015: Google OAuth 2.0 Identity Lifecycle & Anti-CSRF Protection

- **Description:** The system shall support user authentication with Google using OAuth 2.0 with minimal read-only scopes (`gmail.readonly` , `openid` , `userinfo.email` , `userinfo.profile`).
- **Security:** Must generate a cryptographically random, timestamped state token for anti-CSRF validation. All session payloads must be serialized using HMAC SHA-256 signatures via `URLSafeTimedSerializer` and transmitted via HttpOnly cookies.

FR-016: Live Gmail Mailbox Scanning & Threat Categorization

- **Description:** Authenticated users shall be able to fetch up to 50 recent inbox messages from their Google account, execute batch threat analysis, and categorize emails into `"Spam"` , `"Ham"` , or `"Needs Review"` .
- **Output:** Live threat summary cards, review counters, and individual email threat badges.

FR-017: Configurable Uncertainty Review Margin Thresholds

- **Description:** The system shall provide runtime-adjustable threshold boundaries (default: Low = 0.35, High = 0.65). Emails with $\mathrm{P}(\mathrm{Spam}) \geq 0.65$ are marked Spam; ≤ 0.35 are marked Ham; and intermediate values ($0.35 < \mathrm{P} < 0.65$) are flagged as `"Needs Review"` .

FR-018: Pre-Inference Cryptographic Integrity Check

- **Description:** Prior to executing inference, `SpamPredictor` shall compute the SHA-256 hash of `models/spam_pipeline.joblib` . If the hash does not match `c652119c99f0db2abe0788969d002818478bb63be95e31e5f337eda8da6ddbb0` , the system shall immediately abort with `SpamGuardModelError` to prevent model tampering.

---

4. Non-Functional Requirements

NFR-001: Performance — Single Email Latency

- **Requirement:** Median single-email inference execution time (text cleaning, vectorization, Naïve Bayes scoring, and top-5 explainability extraction) shall not exceed 50.0 ms under standard CPU workloads.
- **Empirical Measurement:** Hardware benchmarks demonstrate a median single inference latency of **1.04 ms** (over 45x faster than required threshold).

NFR-002: Performance — Batch Classification Throughput

- **Requirement:** The system shall sustain a batch inference throughput exceeding 5,000 emails per second for text-only email batches on standard x86_64 hardware.
- **Empirical Measurement:** Vectorized batch inference on 1,000 synthetic records executes in **81.2 ms**, representing an effective throughput of **12,315 emails/second**.

NFR-003: Accuracy & Asymmetric Error Control

- **Requirement:** The model shall achieve an overall Classification Accuracy of $\geq 95.0\%$, a Spam Precision of $\geq 95.0\%$, and a False Positive Rate (FPR) of $\leq 2.0\%$ on the held-out test partition ($\mathrm{n}=889$).
- **Empirical Measurement:** Evaluated test performance:
- Accuracy: 97.30%
- Spam Precision: 97.33%

- Spam Recall: 93.75%
- Spam F1-Score: 95.51%
- ROC-AUC: 0.9965
- False Positive Rate: 1.13% (7 False Positives out of 617 legitimate emails).

NFR-004: Reliability & Determinism

- **Requirement:** The classification engine must be 100% deterministic. Submitting identical input text to `predict_email()` must produce identical predicted classes, probabilities, and feature contributions across arbitrary executions.
- **Verification:** Verified by automated test case `test_deterministic_predictions` passing with identical numerical assertions.

NFR-005: Availability & Graceful Degradation

- **Requirement:** In the event that the primary Next.js frontend or FastAPI service experiences disruption, the Streamlit dashboard on port 8501 shall continue functioning independently using direct in-process bindings. If the model artifact is unavailable, the API must return structured HTTP 500 error messages rather than unhandled process terminations.

NFR-006: Security — Denial-of-Service Defense

- **Requirement:** The system shall enforce a strict upper bound of 1,000,000 characters (1 MB) on input email text. Requests exceeding this threshold must be rejected immediately during text preparation with HTTP 422 to protect CPU memory and regex engines.

NFR-007: Security — Formula Injection Neutralization

- **Requirement:** The system must neutralize active formula execution prefixes (`=` , `+` , `-` , `@` , `\t` , `\r` , `\n`) in all CSV export outputs in compliance with OWASP guidelines.
- **Verification:** Verified by unit tests asserting that `=SUM(A1:A10)` is sanitized to `'=SUM(A1:A10)`.

NFR-008: Security — Cryptographic Session & State Signing

- **Requirement:** All session cookies and OAuth state tokens must be cryptographically signed using `URLSafeTimedSerializer` with SHA-256 HMAC. State tokens must expire after 15 minutes; session cookies must expire after 7 days and enforce `HttpOnly` flags.

NFR-009: Privacy — Zero Mail Storage Invariant

- **Requirement:** The system shall adhere to a strict privacy-first architecture. Electronic mail content submitted for inspection must exist solely in ephemeral RAM during inference. The system shall never persist email bodies to SQLite, disk caches, or external third-party cloud logging services.

NFR-010: Maintainability & Automated Test Coverage

- **Requirement:** The software codebase shall maintain modular separation between preprocessing, configuration, inference, presentation, and testing. The automated test suite must achieve $\geq 90\%$ passing rate across all test modules.
- **Empirical Measurement: 152 out of 152 Pytest tests pass (100%)** + **5 out of 5 Node.js frontend tests pass (100%)**, totaling **157 passing automated tests**.

NFR-011: Portability & Cross-Platform Compliance

- **Requirement:** The entire software system must execute without modification across Windows, Linux, and macOS environments using standard Python virtual environments and Node.js package managers. All file path manipulations must use `pathlib.Path` with forward/backward slash normalization.

NFR-012: Usability & Ergonomics

- **Requirement:** Both web interfaces shall support responsive design across Desktop (1920px), Laptop (1366px), Tablet (768px), and Mobile (390px). Threat levels must be clearly delineated using high-contrast, accessible color schemes (Emerald `#54D6A0` for Ham, Crimson `#FF5D66` for Spam, Amber `#F5A623` for Needs Review).

---

5. External Interface Requirements

5.1 User Interfaces

1. **Next.js 16 Editorial Web Workstation (`frontend/`):**
2. Served on `http://localhost:3000` .
3. Built with Next.js 16, React 19, TypeScript, and Tailwind CSS.
4. Features: Collapsible left navigation sidebar, top breadcrumb bar with live telemetry, Overview Dashboard, Single Threat Inspector, Batch CSV Classifier, Model Performance Analytics, Connected Accounts Manager, Uncertainty Settings, and Message Detail Slide-Over Drawer.
5. **Streamlit 1.66 Resilient Dashboard (`app.py`):**
6. Served on `http://localhost:8501` .
7. Built with Streamlit and Plotly Graph Objects.
8. Features: Multi-tab navigation, Plotly probability gauge, confusion matrix heatmap, top discriminative feature word clouds, sample loader dropdown, and sanitized CSV download button.

5.2 Backend API Interface

The FastAPI service exposes the following typed RESTful endpoints: - `GET /api/health` — Returns `SystemHealthResponse` with model availability and SHA-256 verification. - `POST /api/predict` — Ingests `PredictRequest` , returns `PredictResponse` with probabilities and top-k log-odds features. - `POST /api/predict/batch` — Ingests `BatchPredictRequest` , returns `BatchResponse` with aggregate metrics. - `POST /api/predict/csv` — Ingests multipart form CSV file, returns `BatchResponse` . - `GET /api/metrics` — Returns full held-out test evaluation report and baseline comparison data. - `GET /api/samples` — Returns list of pre-configured synthetic sample emails. - `GET /api/auth/status` — Returns Google OAuth authentication status and configured scopes. - `GET /api/auth/google/start` — Generates Google OAuth consent URL with cryptographic state token. - `GET /api/auth/google/callback` — Validates OAuth authorization code, creates session token, sets HttpOnly cookie, and redirects to frontend. - `POST /api/auth/disconnect` — Clears authenticated session and revokes access token. - `GET /api/gmail/messages` — Retrieves recent Gmail message metadata. - `POST /api/gmail/scan/recent` — Scans up to 50 recent inbox emails and returns threat classifications. - `GET /api/settings` & `POST /api/settings` — Retrieves and updates classification uncertainty thresholds.

5.3 Software Interfaces

- **Scikit-learn Pipeline:** In-memory integration via `joblib.load()` .
- **Pandas DataFrames:** Tabular batch data manipulation and CSV parsing.
- **Plotly & Matplotlib Engines:** Rendering confusion matrix heatmaps and ROC curves.

- **Google OAuth 2.0 API:** HTTPS communication with `accounts.google.com` and `gmail.googleapis.com`.

5.4 Hardware Requirements

- **Processor:** Minimum 2.0 GHz dual-core x86_64 or ARM64 processor.
- **Memory (RAM):** Minimum 4 GB RAM (8 GB recommended for large batch processing).
- **Storage:** Minimum 500 MB free disk space for repository, dependencies, and model artifacts.

---

6. Data Requirements

6.1 Data Inputs

- **Single Email Input:**
- `subject`: String, 0 to 1,000,000 characters.
- `body`: String, 0 to 1,000,000 characters.
- At least one field must contain printable, non-whitespace characters.
- **Batch CSV Input:**
- Standard Comma-Separated Values file formatted as UTF-8, UTF-8-BOM, ISO-8859-1, or Windows-1252.
- Header row containing `subject` and `body` (or `text`).
- Row limit: $\leq 10{,}000$ records.

6.2 Data Outputs & Prediction Schema

Every single-email prediction outputs an immutable `PredictionResult` structure:

```
{
  "predicted_class": 1,
  "predicted_label": "Spam",
  "spam_probability": 0.9852,
  "ham_probability": 0.0148,
  "is_spam": true,
  "model_identifier": "MultinomialNB(alpha=0.1) + TF-IDF(20,000 features)",
  "processing_time_ms": 1.24,
  "cleaned_char_count": 412,
  "top_features": [
    {
      "token": "guaranteed",
      "weight": 0.3812,
      "log_odds_diff": 4.1205,
      "contribution": 1.5707,
      "association": "Spam-leaning"
    }
  ]
}
```

6.3 Model Artifact Specifications

- **File Location:** `models/spam_pipeline.joblib`
- **File Size:** Approximately 1.15 MB
- **Expected SHA-256 Checksum:** `c652119c99f0db2abe0788969d002818478bb63be95e31e5f337eda8da6ddbb0`
- **Pipeline Architecture:**

- Step 1: `TfidfVectorizer(max_features=20000, ngram_range=(1, 2), min_df=2, max_df=0.98, sublinear_tf=True)`
- Step 2: `MultinomialNB(alpha=0.1, fit_prior=True)`

6.4 Dataset Partitions (Apache SpamAssassin Public Corpus)

The underlying dataset originates from the official Apache SpamAssassin public archives: - `20030228_easy_ham.tar.bz2` ($n=2,472$ clean) - `20030228_easy_ham_2.tar.bz2` ($n=1,389$ clean) - `20030228_hard_ham.tar.bz2` ($n=248$ clean) - `20030228_spam.tar.bz2` ($n=482$ clean) - `20050311_spam_2.tar.bz2` ($n=1,332$ clean) - **Total Valid Corpus:** 5,923 emails (4,109 Ham [69.37%], 1,814 Spam [30.63%]). - **Stratified Partitioning (Random Seed 42):** - **Train Partition (70%):** 4,145 samples (2,876 Ham, 1,269 Spam). - **Validation Partition (15%):** 889 samples (616 Ham, 273 Spam). - **Held-Out Test Partition (15%):** 889 samples (617 Ham, 272 Spam, Quarantined).

---

7. Use Cases

UC-01: Single Email Threat Inspection

- **Primary Actor:** Security Analyst
- **Preconditions:** FastAPI service running on port 8000; Next.js frontend running on port 3000 (or Streamlit running on port 8501); model authenticated.
- **Main Success Scenario:**
- User navigates to the Single Email Scanner view.
- User enters Subject and Body text (or clicks a synthetic demonstration sample).
- User clicks "Classify Email" (or "Scan Email").
- System normalizes and validates text against empty bounds and size limits.
- System generates TF-IDF sparse vector and computes Naïve Bayes posterior probabilities.
- System renders Threat Verdict Card (Ham Shield or Phishing Alert).
- System renders animated probability gauge displaying $\mathrm{P}(\mathrm{Spam})$ and $\mathrm{P}(\mathrm{Ham})$.
- System renders execution latency telemetry.
- **Alternative Scenarios:**
- *4a. Empty Input:* System halts inference, renders validation banner: "*Email content is empty or contains only whitespace after cleaning*".
- *4b. Oversized Input (> 1MB):* System halts inference, renders security alert: "*Input text exceeds maximum allowed limit of 1,000,000 characters*".
- **Postconditions:** Classification result displayed; zero email text written to persistent disk storage.

UC-02: Feature Contribution and Explainability Inspection

- **Primary Actor:** Security Analyst / Academic Evaluator
- **Preconditions:** UC-01 has executed with `explain=True`.
- **Main Success Scenario:**
- System extracts non-zero feature terms present in the document vocabulary.
- System computes token log-odds difference: $\Delta_{\mathrm{odds}} = \log \mathrm{P}(w \mid \mathrm{Spam}) - \log \mathrm{P}(w \mid \mathrm{Ham})$.
- System calculates token contribution: $\mathrm{Weight} \times \Delta_{\mathrm{odds}}$.
- System sorts tokens by absolute magnitude and extracts top k terms.

- System displays interactive table showing Token, TF-IDF Weight, Log-Odds Difference, Contribution, and Polarity (`Spam-leaning` vs. `Ham-leaning`).
- **Postconditions:** User understands exactly which vocabulary terms caused the classification decision.

UC-03: Batch CSV Scanning and Sanitized Export

- **Primary Actor:** Security Analyst / Mail Administrator
- **Preconditions:** User possesses a CSV file with up to 10,000 emails.
- **Main Success Scenario:**
- User navigates to Batch Scanner view.
- User uploads CSV file via drag-and-drop or file selector.
- System validates CSV header schema for `'subject'` / `'body'` or `'text'`.
- User clicks "Scan Entire Batch".
- System executes vectorized text preparation, vectorization, and model inference across all rows.
- System displays summary metric cards: Total Scanned, Spam Detected, Ham Count, Spam Percentage, Average Latency.
- User inspects filtered results table.
- User clicks "Export Clean Results (CSV)".
- System applies `sanitize_csv_cell()` to neutralize spreadsheet formula injection.
- Browser downloads sanitized CSV file.
- **Alternative Scenarios:**
- *3a. Invalid Schema:* CSV lacks required columns; system displays schema error with required format guide.
- *3b. Exceeds Row Limit (> 10,000 rows):* System rejects upload with batch size bound error.
- **Postconditions:** Sanitized classification report saved to user's local downloads.

UC-04: Model Analytics and Confusion Matrix Review

- **Primary Actor:** Academic Evaluator
- **Preconditions:** Test evaluation report `reports/test_evaluation.json` exists.
- **Main Success Scenario:**
- User navigates to Model Analytics view.
- System reads frozen evaluation figures and renders interactive KPI cards (97.30% Accuracy, 97.33% Precision, 93.75% Recall, 95.51% F1, 0.9965 ROC-AUC).
- System renders Confusion Matrix Heatmap ($\mathrm{TN}=610$, $\mathrm{FP}=7$, $\mathrm{FN}=17$, $\mathrm{TP}=255$).
- System displays comparative bar chart benchmarking SpamGuard AI against the Zero-Intelligence Majority Baseline.
- System renders Top Discriminative Words bar charts for Spam and Ham.
- **Postconditions:** Evaluator verifies all empirical academic claims without modifying model weights.

UC-05: Live Gmail Mailbox Scanning

- **Primary Actor:** Security Analyst / End User
- **Preconditions:** Google OAuth credentials configured in `.env`.
- **Main Success Scenario:**
- User navigates to Connected Accounts and clicks "Connect Google Account".
- System generates OAuth 2.0 authorization URL with anti-CSRF state token.

- User approves read-only permissions on Google consent screen.
- Google redirects to `/api/auth/google/callback`.
- System exchanges code for tokens, sets signed session cookie, and redirects user to `/inbox`.
- User selects batch count and clicks "Scan Recent Inbox".
- System fetches messages, parses plain text / HTML fallback, and executes threat inference.
- System flags emails exceeding threshold as Spam or Needs Review.
- **Postconditions:** Mailbox scanned; zero email bodies stored on server.

UC-06: System Health and Model Integrity Audit

- **Primary Actor:** System Administrator / Automated Monitor
- **Preconditions:** FastAPI backend running.
- **Main Success Scenario:**
- Client sends `GET /api/health`.
- System computes real-time SHA-256 hash of `models/spam_pipeline.joblib`.
- System compares calculated hash with `c652119c99f0db2abe0788969d002818478bb63be95e31e5f337eda8da6ddbb0`.
- System responds with HTTP 200 and JSON containing `"status": "OPERATIONAL"`, `"model_sha256_verified": true`.
- **Alternative Scenarios:**
- *2a. Model Corrupted or Missing:* System responds with `"status": "DEGRADED"`, `"model_sha256_verified": false`.
- **Postconditions:** Telemetry confirmed.

---

8. Requirement Traceability Matrix (Summary)

Req ID	Requirement Summary	Implementing Module	Related Use Case	UML Diagram	Test Case ID	Status
FR-001	Email Subject & Body Entry	`src/predict.py`, `EmailScanner.tsx`, `app.py`	UC-01	01, 03, 04	`TC-POS-01`, `TC-FNC-01`	**PASS**
FR-002	Synthetic Sample Selection	`src/ui_helpers.py`, `src/api.py`	UC-01	01, 03	`TC-FNC-02`	**PASS**
FR-003	Preprocessing & Empty Validation	`src/preprocess.py`, `src/predict.py`	UC-01	03, 04, 05	`TC-NEG-01`, `TC-BND-01`	**PASS**
FR-004	Binary Threat Classification	`src/predict.py`, `models/spam_pipeline.joblib`	UC-01	01, 02, 03, 04	`TC-POS-01`, `TC-POS-02`	**PASS**
FR-005	Calibrated Probability Scoring	`src/predict.py`, `src/api.py`	UC-01	01, 03, 04	`TC-POS-01`, `TC-FNC-03`	**PASS**
FR-006	Feature Log-Odds Explainability	`src/predict.py` (`explain_email`)	UC-02	01, 02, 03, 04	`TC-FNC-04`, `TC-INT-02`	**PASS**
FR-007	CSV Upload & Schema Validation	`src/predict.py`, `src/api.py`, `BatchScanner.tsx`	UC-03	01, 04, 05	`TC-NEG-02`, `TC-BND-02`	**PASS**
FR-008	Vectorized Batch Inference	`src/predict.py` (`predict_batch`)	UC-03	01, 02, 05	`TC-FNC-05`, `TC-INT-03`	**PASS**
FR-009	Batch Result Filtering	`BatchScanner.tsx`, `app.py`	UC-03	01, 05	`TC-FNC-06`	**PASS**
FR-010	CSV Injection Defense & Export	`src/predict.py` (`sanitize_csv_cell`)	UC-03	01, 04	`TC-ERR-02`, `TC-SEC-01`	**PASS**
FR-011	Model Performance Analytics	`src/evaluate.py`, `ModelAnalytics.tsx`	UC-04	01, 02	`TC-FNC-07`	**PASS**
FR-012	API Health & Telemetry	`src/api.py` (`/api/health`)	UC-06	01, 02	`TC-INT-01`	**PASS**
FR-013	RESTful Prediction Endpoints	`src/api.py` (`/api/predict`)	UC-01, UC-03	02, 03	`TC-INT-02`, `TC-INT-03`	**PASS**
FR-014	Dual Application Architecture	`app.py`, `frontend/`, `src/api.py`	UC-01	01, 02	`TC-INT-04`	**PASS**

Req ID	Requirement Summary	Implementing Module	Related Use Case	UML Diagram	Test Case ID	Status
FR-015	Google OAuth 2.0 Identity Flow	`src/gmail/auth.py`, `src/api.py`	UC-05	01, 02	`TC-SEC-02`, `TC-SEC-03`	**PASS**
FR-016	Live Gmail Mailbox Scanner	`src/gmail/scanner.py`, `GmailInboxView.tsx`	UC-05	01, 02	`TC-FNC-08`	**PASS**
FR-017	Uncertainty Margin Thresholds	`src/api.py`, `SettingsView.tsx`	UC-05	01, 05	`TC-FNC-09`	**PASS**
FR-018	SHA-256 Checksum Verification	`src/predict.py` (`compute_file_sha256`)	UC-06	01, 04, 05	`TC-ERR-01`, `TC-SEC-04`	**PASS**
NFR-001	Latency ≤ 50 ms	`src/predict.py`	UC-01	03, 04	`TC-BND-03`	**PASS**
NFR-002	Throughput $>$ 5,000 emails/sec	`src/predict.py`	UC-03	05	`TC-BND-04`	**PASS**
NFR-003	Accuracy $\geq 95\%$, FPR $\leq 2\%$	`models/spam_pipeline.joblib`	UC-04	02	`TC-FNC-07`	**PASS**
NFR-006	Payload Cap 1,000,000 chars	`src/predict.py`	UC-01	03, 04, 05	`TC-BND-05`	**PASS**
NFR-007	Formula Injection Prefixing	`src/predict.py`	UC-03	04	`TC-ERR-02`	**PASS**
NFR-008	Anti-CSRF & Signed Cookies	`src/gmail/auth.py`	UC-05	02	`TC-SEC-03`	**PASS**
NFR-009	Zero Mail Storage on Disk	System Wide	UC-01, UC-03	04, 05	`TC-SEC-05`	**PASS**
NFR-010	Automated Test Coverage $\geq 90\%$	`tests/`, `frontend/tests/`	System Wide	All	`TC-QA-01`	**PASS**

9. Document References

1. **Apache SpamAssassin Public Corpus:** Apache Software Foundation. *SpamAssassin Public Corpus Archives (2003-2005)*. Available at: `https://spamassassin.apache.org/old/publiccorpus/`.
2. **Scikit-learn Documentation:** Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
3. **Multinomial Naïve Bayes Foundations:** McCallum, A., & Nigam, K. (1998). *A comparison of event models for Naive Bayes text classification*. AAAI-98 workshop on learning for text categorization, 752, 41-48.
4. **IEEE Std 830-1998:** IEEE Computer Society. *IEEE Recommended Practice for Software Requirements Specifications*. IEEE, New York.

5. **OWASP Top 10 — CSV Injection Defense:** Open Web Application Security Project (OWASP). *CSV Formula Injection Mitigation Guidelines (2024)*.
6. **RFC 6265:** Internet Engineering Task Force (IETF). *HTTP State Management Mechanism (Cookies)*. RFC Editor, 2011.
7. **NIST Special Publication 800-63B:** National Institute of Standards and Technology. *Digital Identity Guidelines: Authentication and Lifecycle Management*. U.S. Department of Commerce.
8. **SpamGuard AI Project Documentation:**
9. Architecture Specifications: `docs/architecture.md`
10. Final Project Report: `docs/SpamGuard_AI_Final_Project_Report.md`
11. Model Evaluation Findings: `reports/evaluation_findings.md`
12. Verified Test Evaluation Artifact: `reports/test_evaluation.json`